

DAY-69

1. Surrounded Regions:-

IP
X X X X
O O O X
X O X X
X X X X

OP
X X X X
O O O X
X O X X
X X X X

→

O O O X
X O X X
X X X X

Can't convert because
is connected
to border O

IP

X X X X
X O O X
X X O X
X O X X

→

X X X X
X X X X
X X X X
X O X X

O O
X O → not
connected to border.

Border 'O' = escape source

Any O reachable from it = safe zone.

everything else Captured.

a. Mark all O's connected to border ('T')

b. Flip remaining O's → X

c. Convert T → O.


```
def solve(self, board):
```

```
    if not board or not board[0]:
```

```
        return
```

```
    rows, cols = len(board), len(board[0])
```

```
    def dfs(r, c):
```

```
        if r < 0 or c < 0 or r > rows or c > cols
```

```
            or board[r][c] != 'O':
```

```
            return
```

```
        board[r][c] = 'T'
```

```
        dfs(r+1, c)
```

```
        dfs(r-1, c)
```

```
        dfs(r, c+1)
```

```
        dfs(r, c-1)
```

① // run dfs from all border cells because 'O' ^{being} on any cell.

```
    for r in range(rows):
```

```
        dfs(r, 0) // check in each row in first col
```

```
        dfs(r, cols-1) // check in each row in last col
```

```
    for c in range(cols):
```

```
        dfs(0, c) // top row
```

```
        dfs(rows-1, c) // bottom row
```

② // flip O → X and X → O

```
    for r in range(rows):
```

```
        for c in range(cols):
```

```
            if board[r][c] == 'O':
```

```
                board[r][c] = 'X'
```

```
            if board[r][c] == 'X':
```

```
                board[r][c] = 'O'
```


2. Pacific Atlantic Water Flow:-

Find all cells in a matrix for which water can flow to both Pacific & Atlantic oceans.

Key insight:-

instead of checking every cell $\rightarrow$ clean path (slow)

a. Start from oceans & see where water can flow inwards.

- Pacific touches: top row + left column

- Atlantic touches: bottom row + right column.

```
def PacificAtlantic(self, heights):
```

```
    if not heights or not heights[0]:
```

```
        return []
```

```
    rows, cols = len(heights), len(heights[0])
```

```
    pacific = set()
```

```
    atlantic = set()
```

```
    def dfs(r, c, visited, prevHeight):
```

```
        if r < 0 or c < 0 or r > rows or c > cols or
```

```
            (r, c) in visited or heights[r][c] < prevHeight:
```

```
            return
```

```
        visited.add((r, c))
```

```
        dfs(r+1, c, visited, heights[r][c])
```

```
        dfs(r-1, c, visited, heights[r][c])
```

```
        dfs(r, c+1, visited, heights[r][c])
```

```
        dfs(r, c-1, visited, heights[r][c])
```

// step 1:- Run dfs from pacific borders

```
for r in range(rows):
```

```
    dfs(r, 0, pacific, heights[r][0]) // left col
```

```
for c in range(cols):
```

```
    dfs(0, c, atlantic, heights[0][c]) // top row.
```


// Step 2: Run dfs from atlantic borders

for r in range(rows):

dfs(r, cols-1, atlantic, heights[r][cols-1]) // right col

for c in range(cols):

dfs(rows-1, c, atlantic, heights[rows-1][c]) // bottom^{row}

// Step 3: Intersection cells reachable from both oceans
return list(pacific & atlantic)

ex: pacific = {(0,0), (0,1), (1,0)}

atlantic = {(0,1), (1,0), (1,1)}

list(pacific & atlantic) = [(0,1), (1,0)]

a = set(['a', 'b', 'c'])

print(a) // {'a', 'b', 'd'}

visited.add(r,r) = pacific.add(r,c)

Since pacific is passed to method.

because visited is a reference to pacific set.

o/p = [(4,0), (0,4)] return list(pacific & atlantic)

o/p = [(4,0), (0,4)] return [list(cell) for cell in (pacific & atlantic)]

3. Rotting oranges

Return min minutes to rot all oranges or -1 if impossible.

approach (multi-source BFS)

Start bfs from all rotten oranges at once.

def orangesRotting(grid):

if not grid or not grid[0]:

return -1

rows, cols = len(grid), len(grid[0])

queue = deque()

fresh = 0

for r in range(rows):

for c in range(cols):

if grid[r][c] == 2:

queue.append((r, c))

elif grid[r][c] == 1:

fresh += 1

directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

minutes = 0

while queue and fresh > 0:

for _ in range(len(queue)):

r, c = queue.popleft()

for dr, dc in directions:

nr, nc = r + dr, c + dc

if 0 ≤ nr < rows and 0 ≤ nc < cols

and grid[nr][nc] == 1:

grid[nr][nc] = 2

queue.append((nr, nc))

fresh -= 1

minutes if fresh == 0 else -1

minutes += 1